

Quantum Computing: Ringing the Bell States

In this fifth article in the series on quantum computing, we take a closer look at entanglement, meet the four Bell states, and examine the many ways in which a real superconducting quantum computer can go wrong.



In the previous article in this series (OSFY, September 2026), we finally left the comfortable world of the single qubit behind. We met the Controlled-NOT (CNOT) gate, our first multi-qubit gate, and used it to create a pair of entangled qubits. We also learned how to make Qiskit's simulator misbehave on purpose by attaching a noise model to it, and we executed a quantum program on an actual superconducting quantum computer through the IBM Quantum Platform. At the end of that article, three promises were made: to look more deeply into entanglement, to introduce the Bell states, and to take a closer look at the various sources of error that affect today's superconducting quantum computers. This article keeps all three promises.

The central theme of this article is quantum entanglement and, in particular, a family of four two-qubit states known as the Bell states. If superposition is what makes a single qubit interesting, entanglement is what makes a collection

of qubits powerful. Erwin Schrödinger, who coined the term entanglement in 1935, went considerably further than that. He described it not as one interesting feature of quantum mechanics among many, but as the one feature that forces quantum mechanics to depart completely from classical ways of thinking. In the previous article we produced an entangled pair almost casually, with two lines of Qiskit code. In this article we shall find out what we actually produced.

As in the previous articles, we shall remain faithful to the three-level approach of this series and explore entanglement through code, mathematics and hardware. But before meeting the Bell states, let us honour a small tradition of this series: we shall first send our code to a real quantum computer.

Entanglement on real hardware

In the previous article, we executed the program *qiskit8.py*, a single-qubit Pauli-X circuit, on the IBM quantum processor *ibm_marrakesh*. That circuit involved just one qubit and one gate, and it was about as gentle a request as one can make of a quantum computer. Let us now raise the stakes. The program *qiskit10.py*, shown below, takes the two-qubit entangling circuit of *qiskit9.py* from the previous article—a Hadamard gate followed by a CNOT gate—and executes it on real superconducting hardware instead of the *AerSimulator*.

```
1. from qiskit import QuantumCircuit, transpile
2. from qiskit_ibm_runtime import QiskitRuntimeService,
   SamplerV2 as Sampler
3. API_KEY = "PasteYourAPIKeyHere"
4. QiskitRuntimeService.save_account(
   channel="ibm_quantum_platform",
   token=API_KEY,
   set_as_default=True,
   overwrite=True)
5. service = QiskitRuntimeService( )
6. backend = service.least_busy(operational=True,
```